Assignment No. 11

Content Beyond Syllabus: 1

Aim:

Analyze the performance of your profile page (Experiment 1) using browser developer tools and apply optimizations like lazy loading and minification.

Software / Libraries Required

- Web Browser (Google Chrome / Mozilla Firefox)
- Browser Developer Tools
- Code Editor (VS Code / Notepad++)
- HTML5, CSS3, JavaScript
- Minification tools or online compressors

Objectives

- To analyze the performance of a webpage using browser developer tools
- To identify factors affecting page load time and user experience
- To learn and implement lazy loading for optimizing image loading
- To understand and apply minification techniques for HTML, CSS, and JavaScript
- To improve overall website performance by reducing resource size and load time

Theory

Website performance is an important factor in web development because it directly affects **user experience, page load time, and search engine ranking**. A slow-loading page can lead to poor user engagement and higher bounce rates. To improve performance, developers analyze web pages using **Browser Developer Tools** and apply optimization techniques such as **lazy loading and minification**.

In this experiment, the performance of the **Profile Page developed in Experiment 1** is analyzed and optimized using browser tools.

Browser Developer Tools

Modern browsers such as **Google Chrome, Firefox, and Edge** provide **Developer Tools** that help developers inspect and analyze web pages.

Developer tools can be opened by:

Right Click → Inspect

or

Press F12

These tools provide several panels such as:

- Elements
- Console
- Network
- Performance
- Lighthouse

These panels help identify performance issues such as **large images, slow scripts, and unnecessary files**.

Performance Analysis

The **Performance Panel** records how the page loads and displays information about:

- Page loading time
- Rendering time
- JavaScript execution
- Network requests

Steps to analyze performance:

1. Open the profile page in the browser.
2. Open **Developer Tools (F12)**.
3. Go to the **Performance or Network tab**.

4. Reload the page to record the loading process.
5. Analyze loading time, file size, and number of requests.

Key metrics observed:

- **Load Time**
- **First Contentful Paint (FCP)**
- **Total Blocking Time**
- **Largest Contentful Paint (LCP)**

Lazy Loading

Lazy Loading is an optimization technique in which images or content are loaded **only when they are needed**, rather than loading everything when the page first opens.

This improves performance because:

- Reduces initial page load time
- Saves bandwidth
- Loads images only when the user scrolls

Example of Lazy Loading in HTML:

```
<img src="profile.jpg" loading="lazy" alt="Profile Image">
```

In this example, the image will load only when it becomes visible on the screen.

Advantages:

- Faster page loading
- Reduced server load
- Improved user experience

Minification

Minification is the process of removing unnecessary characters from code such as:

- Spaces

- Line breaks
- Comments

This reduces file size and improves loading speed.

Example:

Normal JavaScript

```
function greet() {

    console.log("Welcome to Profile Page");

}
```

Minified JavaScript

```
function greet(){console.log("Welcome to Profile Page");}
```

Minification can be applied to:

- **HTML**
- **CSS**
- **JavaScript**

Benefits:

- Smaller file size
- Faster downloads
- Improved performance

Network Optimization

Using the **Network tab**, developers can check:

- File size
- Loading time
- Number of requests

- Resource types (CSS, JS, Images)

Common improvements include:

- Compressing images
- Reducing HTTP requests
- Using caching
- Combining CSS and JS files

Application of Optimizations

In this experiment, the following optimizations were applied to the profile page:

1. **Lazy loading** was added to images to load them only when required.
2. **Minification** was applied to CSS and JavaScript files to reduce file size.
3. Large images were compressed to reduce loading time.
4. Unnecessary scripts and styles were removed.

These optimizations improved the page performance and reduced loading time.

Algorithm

Start
Open profile webpage in browser
Open Developer Tools and inspect network performance
Identify large images or slow-loading files
Apply lazy loading to images
Minify CSS and JavaScript files
Reload page and measure performance improvement
Compare loading time before and after optimization
Stop

Oral Questions

- What is webpage performance optimization?
- What is lazy loading?

- What is minification?
- How do developer tools help analyze performance?
- What factors slow down webpage loading?
- **Conclusion**

In this experiment, the performance of the profile webpage was successfully analyzed using browser developer tools. Various issues like large image sizes and unnecessary files were identified. By applying optimization techniques such as lazy loading and minification, the page load time was reduced and performance was improved. This experiment helped in understanding how optimization techniques enhance user experience and website efficiency.

Assignment No. 12

Content Beyond Syllabus: 2

Aim:

Use the Java Streams API to perform complex operations on a list of objects (e.g., group students by marks, find the average mark, or partition into passing/failing).To perform advanced data processing operations using Java Streams API to group objects, calculate averages, and partition data efficiently.

Software / Libraries Required

- Java JDK 8 or above
- IDE (Eclipse / IntelliJ / VS Code)
- Java Collections Framework

Objectives

- To understand and use Java Streams API for data processing
- To perform operations like grouping, averaging, and filtering on collections
- To implement partitioning of data into categories (pass/fail)
- To learn functional programming concepts using lambda expressions
- To write concise and efficient Java code

Theory

The **Java Streams API**, introduced in **Java 8**, provides a modern and powerful approach for

processing collections of data in a functional and efficient way. It allows developers to perform complex operations on data such as filtering, sorting, grouping, and aggregation without writing lengthy loops or complex logic. A stream represents a sequence of elements from a data source such as a list, array, or collection and supports various operations that can be chained together to produce a desired result. The Streams API follows a functional programming style where operations are expressed as a sequence of transformations on the data. This approach makes programs more readable, concise, and easier to maintain. In traditional programming, developers often use loops and conditional statements to process data stored in collections, which can lead to verbose and less maintainable code. However, with the Streams API, many of these operations can be performed using built-in methods and lambda expressions, significantly simplifying the code structure.

In Java Streams, operations are generally divided into two main categories: **intermediate operations and terminal operations**. Intermediate operations transform a stream into another stream and allow multiple processing steps to be chained together. Examples of intermediate operations include filter(), map(), sorted(), and distinct(). These operations do not execute immediately; instead, they build a pipeline of operations that will be applied when the terminal operation is invoked. Terminal operations, on the other hand, produce the final result of the stream processing. Examples of terminal operations include collect(), forEach(), count(), reduce(), and average(). Once a terminal operation is executed, the stream processing pipeline is completed and the result is produced. This mechanism makes streams efficient because operations are performed only when required.

In this experiment, the Java Streams API is used to perform advanced data processing operations on a list of student objects. Each student object contains attributes such as student ID, name, and marks. The first operation performed is **grouping students based on their marks or another attribute**. This is achieved using the Collectors.groupingBy() method, which organizes elements of the stream into groups based on a specific classification function. For example, students can be grouped according to their marks or grades, allowing developers to analyze data more effectively. Grouping is particularly useful in applications where data needs to be categorized or summarized according to certain criteria.

Another important operation demonstrated in this experiment is **calculating the average marks of students**. The Streams API provides built-in methods such as Collectors.averagingDouble() that allow developers to compute the average value of numerical attributes easily. Instead of manually iterating through the list and calculating the sum and average using loops, the Streams API performs this task in a more concise and efficient way.

This method processes all elements in the stream and calculates the average value of the specified field, providing a quick way to obtain statistical information from the data.

The third operation performed is **partitioning the student data into passing and failing categories**. Partitioning is a special type of grouping where elements are divided into two groups based on a given condition. This can be implemented using the Collectors.partitioningBy() method. In this case, students can be divided into two groups depending on whether their marks are greater than or equal to a passing threshold, such as 40 marks. The result of this operation is typically a map containing two lists: one for students who passed and another for students who failed. This approach simplifies decision-making processes and allows developers to quickly analyze subsets of data.

One of the major advantages of the Java Streams API is that it **does not modify the original data source**. Instead, it processes the data and produces a new result while keeping the original collection unchanged. This ensures data integrity and prevents unintended side effects in the program. Additionally, streams support **parallel processing**, which allows operations to be performed concurrently on multiple processors. This feature can significantly improve performance when dealing with large datasets, as tasks can be divided and executed simultaneously.

The Streams API also promotes the use of **lambda expressions and functional interfaces**, which are key features of modern Java programming. Lambda expressions provide a concise way to represent anonymous functions and are commonly used in stream operations to define the behavior of filtering, mapping, or grouping functions. This reduces boilerplate code and makes the program more expressive. For example, conditions such as checking whether a student has passed can be written in a single line using a lambda expression, making the code both simple and readable.

Overall, the Java Streams API offers a flexible and efficient framework for performing complex data processing operations on collections. By using stream operations such as grouping, averaging, and partitioning, developers can analyze and manipulate data in a structured and concise manner. This approach not only improves the readability of the code but also enhances its performance and scalability. In modern Java applications, the Streams API has become an essential tool for handling large volumes of data and implementing functional programming techniques. Therefore, understanding and applying the Streams API is an important skill for Java developers, as it enables them to write cleaner, more efficient, and more maintainable programs.

Algorithm

Start

Create list of student objects

Convert collection to stream

Apply grouping operation based on marks

Calculate average marks

Partition students into pass and fail categories

Display results

Stop

Conclusion

In this experiment, the Java Streams API was used to perform advanced operations on student data such as grouping, averaging, and partitioning. The use of streams simplified the code and improved readability and efficiency. It demonstrated how functional programming concepts can be applied in Java for better data processing.